Posets

Complete Notes for Competitive Programming

---

1. Core Definitions

Relation

A **relation** R on a set S is a subset of S x S. We write a <= b to mean (a, b) in R.

Poset

A **partially ordered set (poset)** P = (P, <=) is a set P with a relation <= satisfying:

1. `Reflexivity: for all a in P, a <= a`
2. `Antisymmetry: a <= b and b <= a => a = b`
3. `Transitivity: a <= b and b <= c => a <= c`

The word "partial" means some pairs may be **incomparable** — neither a <= b nor b <= a.

Comparable, Incomparable, Total Order

x, y are **comparable** if x <= y or y <= x. Otherwise **incomparable**.

A poset where every pair is comparable is a **total order**.

The Three Canonical Examples

Poset	Relation	Incomparable pairs?
(Z, <=)	Standard <=	No — total order
(P(X), subset)	Subset inclusion	Yes — e.g. {1} and {2}
(N, |)	Divisibility	Yes — distinct primes

Antisymmetry vs Symmetry

Property	Statement	Example
Symmetric	a <= b => b <= a	"is a sibling of"
Antisymmetric	a<=b and b<=a => a=b	(Z, <=)

Antisymmetry says: two-way relations force equality. It does NOT say two-way is impossible.

2. Infimum and Supremum

Definitions

Let (P, <=) be a poset and S subset of P.

```
s is a lower bound of S if for all a in S, s <= a
s is an upper bound of S if for all a in S, a <= s
inf(S) = greatest lower bound (if it exists)
sup(S) = least upper bound (if it exists)
```

Meet and Join Notation

```
x ^ y = inf{x, y} (meet, greatest lower bound)
x v y = sup{x, y} (join, least upper bound)
```

Critical point: x ^ y does NOT have to be x or y. It can be a completely different element.

Example in (P({1,2,3}), subset):

```
{1,2} ^ {2,3} = {2} = {1,2} intersect {2,3}
```

The meet is {2} — neither of the original sets.

The Pattern Across Posets

Poset	Meet (^)	Join (v)
(Z, <=)	min	max
(P(X), subset)	intersection	union
(N, |)	gcd	lcm

Minimum and Maximum

```
bottom is the minimum of P if bottom <= x for all x in P
top is the maximum of P if x <= top for all x in P
```

Proposition: If minimum (or maximum) exists, it is unique.

Proof: Suppose bottom_1 and bottom_2 are both minima. Since bottom_1 is a minimum: bottom_1 <= bottom_2. Since bottom_2 is a minimum: bottom_2 <= bottom_1. By antisymmetry: bottom_1 = bottom_2.

3. Lattices

Definition

A poset (P, <=) is a **lattice** if for every pair x, y in P, both x ^ y and x v y exist.

> *A general poset does NOT guarantee this. A lattice does — for every pair.*

Commutativity

```
x ^ y = y ^ x x v y = y v x
```

Proof sketch: {x, y} and {y, x} are the same set, so their glb and lub are the same.

Associativity

```
x ^ (y ^ z) = (x ^ y) ^ z
```

Proof: Let t = x ^ (y ^ z) and s = (x ^ y) ^ z.

t is a lower bound for x and y^z, so t <= x and t <= y^z. Since y^z <= y and y^z <= z, transitivity gives t <= y and t <= z. Since t <= x and t <= y, we get t <= x^y. Since t <= x^y and t <= z, we get t <= s.

By symmetric argument s <= t. By antisymmetry s = t.

Distributive Lattice

```
x v (y ^ z) = (x v y) ^ (x v z)
```

```
x ^ (y v z) = (x ^ y) v (x ^ z)
```

> *Not every lattice is distributive. This is an additional property.*

The Hierarchy

```
Poset ⊃ Lattice ⊃ Distributive lattice ⊃ Boolean algebra
```

4. Chains, Antichains, Dilworth

Chain

A **chain** is a subset C of P where every two elements are comparable:

```
for all a, b in C: a <= b or b <= a
```

Antichain

An **antichain** is a subset A of P where no two elements are comparable:

```
for all a, b in A: a not<= b and b not<= a
```

Example in (P({1,2,3}), subset)

```
Chain: {} subset {1} subset {1,2} subset {1,2,3}
Antichain: {1}, {2}, {3} (no set contains another)
```

CP Translation on Arrays

Define poset on array a[] where a_i <= a_j iff i < j AND a_i <= a_j:

Poset concept	CP meaning
Chain	Increasing subsequence
Antichain	Set of mutually incomparable elements
Longest chain	LIS (Longest Increasing Subsequence)
Largest antichain	Set where no element can extend another

Height and Width

```
Height of P = length of longest chain = LIS length in array poset
Width of P = size of largest antichain
```

Dilworth's Theorem

```
minimum chains to PARTITION P = width(P)
```

Lower bound (obvious): Largest antichain has width(P) incomparable elements. No two can share a chain. So at least width(P) chains needed.

Upper bound (the theorem): You can always achieve exactly width(P) chains.

CP consequence: Minimum number of non-decreasing subsequences to partition an array = length of longest strictly decreasing subsequence.

Appears in problems phrased as: minimum piles, minimum groups, partition into minimum number of X.

Example: Array [3,1,4,1,5,9,2,6]. Longest strictly decreasing subsequence: length 2 (e.g. [9,2]). Minimum chain partition = 2.

5. DAG Correspondence

Poset = DAG

Poset concept	Graph concept
Element	Node
a <= b	Directed edge a -> b
Transitivity	Reachability
Antisymmetry + Transitivity	No cycles (DAG)

Why antisymmetry + transitivity => no cycles: If cycle a->b->c->a exists, transitivity gives a <= c. Then a <= c and c <= a gives a = c by antisymmetry — contradiction.

Topological Sort = Linear Extension

A **linear extension** is a total ordering of all elements consistent with the partial order:

```
a <= b in poset => a comes before b in the total order
```

This is exactly **topological sort** on the DAG.

Why DAG DP Works

Every DP on a DAG processes nodes in topological order — a linear extension of the underlying poset. This guarantees all dependencies of a node are processed before the node itself. The poset structure is why a consistent processing order always exists.

6. Bitmask DP (Practical)

The Rule

```
n <= 20 => think subsets => bitmask DP
```

Represent each subset of {0,...,n-1} as an integer from 0 to 2^n-1.

Bitmask Operations

Operation	Code
A subset of B	(A & B) == A
A intersect B	A & B
A union B	A | B
A minus {i}	A ^ (1 << i) (if i in A)
Add element i	A | (1 << i)
Check if i in A	(A >> i) & 1
Enumerate subsets of A	for (S=A; S>0; S=(S-1)&A)

Subset DP Pattern

```
// dp[S] = answer for subset S
// Process subsets in increasing order of size
for (int S = 0; S < (1 << n); S++) {
    for (int i = 0; i < n; i++) {
        if ((S >> i) & 1) {
            // transition: dp[S] from dp[S ^ (1 << i)]
        }
    }
}
```

SOS DP (Sum over Subsets)

Compute f[S] = sum of a[T] for all T subset of S.

```
Naive: O(3^n) SOS: O(n * 2^n)
```

```
// Initialize f[S] = a[S]
for (int i = 0; i < n; i++)
    for (int S = 0; S < (1 << n); S++)
        if ((S >> i) & 1)
            f[S] += f[S ^ (1 << i)];
```

Common CP Patterns

Constraint	Structure	Algorithm
n <= 20, choose subsets	2^n states	Bitmask DP
Sum over subsets of mask S	Sum over subsets	SOS DP O(n * 2^n)
TSP-style problem	dp[visited][last]	Held-Karp O(2^n * n^2)
Partition into groups	dp[assigned]	Subset DP O(3^n)

7. CP Pattern Summary

What you see in problem	Math structure	Algorithm
Longest increasing subsequence	Longest chain in array poset	DP O(n log n)
Minimum piles / groups	Dilworth: min chain = max antichain	LIS of reversed array
Process in dependency order	Linear extension of DAG	Topological sort
n <= 20, choose subsets	Boolean lattice 2^n	Bitmask DP
Sum over all subsets of mask	SOS over Boolean lattice	SOS DP O(n*2^n)

8. Quick Proof Toolkit

To prove x^y = x^z: Show each is a lower bound for the other, use antisymmetry.

To prove a relation is a poset: Check reflexivity, antisymmetry, transitivity explicitly.

To prove a set is a chain: Show every pair is comparable.

To prove a set is an antichain: Show every pair is incomparable.

To apply Dilworth: Identify the partial order, find the largest antichain, conclude minimum chain partition equals that size.